

USB 打印+HID 复合设备驱动开发调试

版 本 号：V1.0.00

目 录

1 术语与缩略语	3
2 开发需求	4
2.1 软件功能	4
2.2 模块框架	4
3 软件架构	5
3.1 数据流控制	5
3.2 技术原理	6
4 驱动开发	8
4.1 HID+printer 复合功能配置	8
4.2 驱动调试	11

1 术语与缩略语

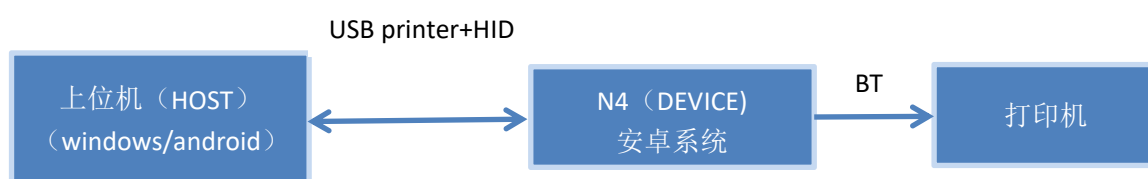
术语及缩写	释义
AP	Application Processor， 安卓应用端， POS 应用中安全无关的应用放在此端。
MCU	Micro controller Unit 微控制器， 负责处理外围器件按键、GPIO、USB 通讯
N4	支持扫码 Android 设备
BT	Bluetooth， 用于设备间数据传输的通讯协议， 用于 N4 来发送打印数据

2 开发需求

2.1 软件功能

基于 N4 终端设备，通过 USB 接口连接客户上位机设备，处理 USB 主控之间的连接和数据交换，完成上位机系统扫码支付和打印小票功能。

2.2 模块框架



模块说明：

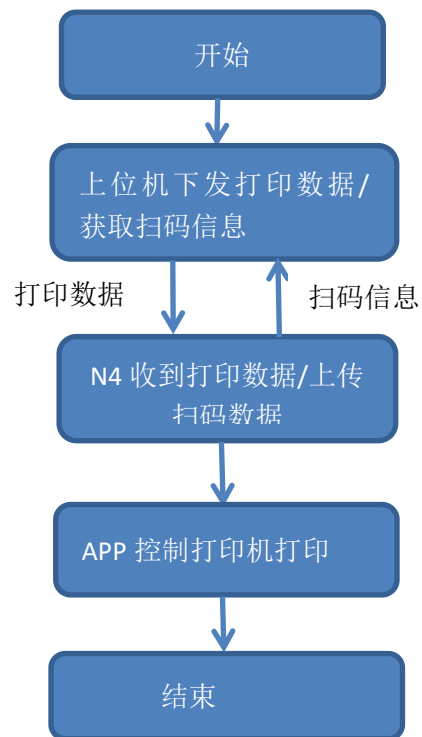
a)、上位机设备（android/windows）做主控,N4 模拟成 USB printer+HID 复合设备，通过 USB 接口和上位机进行数据通讯。

b)、N4 接到上位机打印信息，应用 APP 通过蓝牙接口转发打印数据给打印机。

c)、N4 扫码信息，通过 HID 按键接口传输扫码数据给上位机。

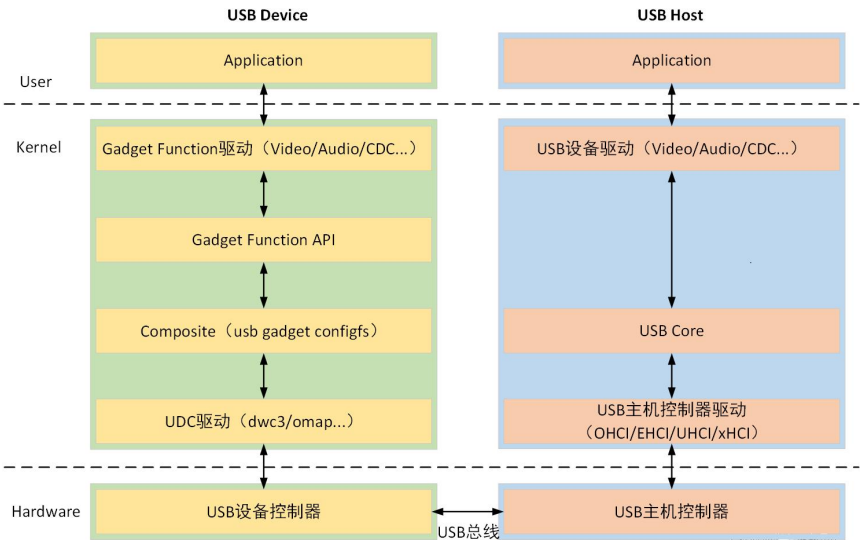
3 软件架构

3.1 数据流控制



3.2 技术原理

基于 USB gadget 架构，N4 配置成 gadget printer 功能设备和上位机进行数据通讯，通过配置 printer 类设备来定制开发。



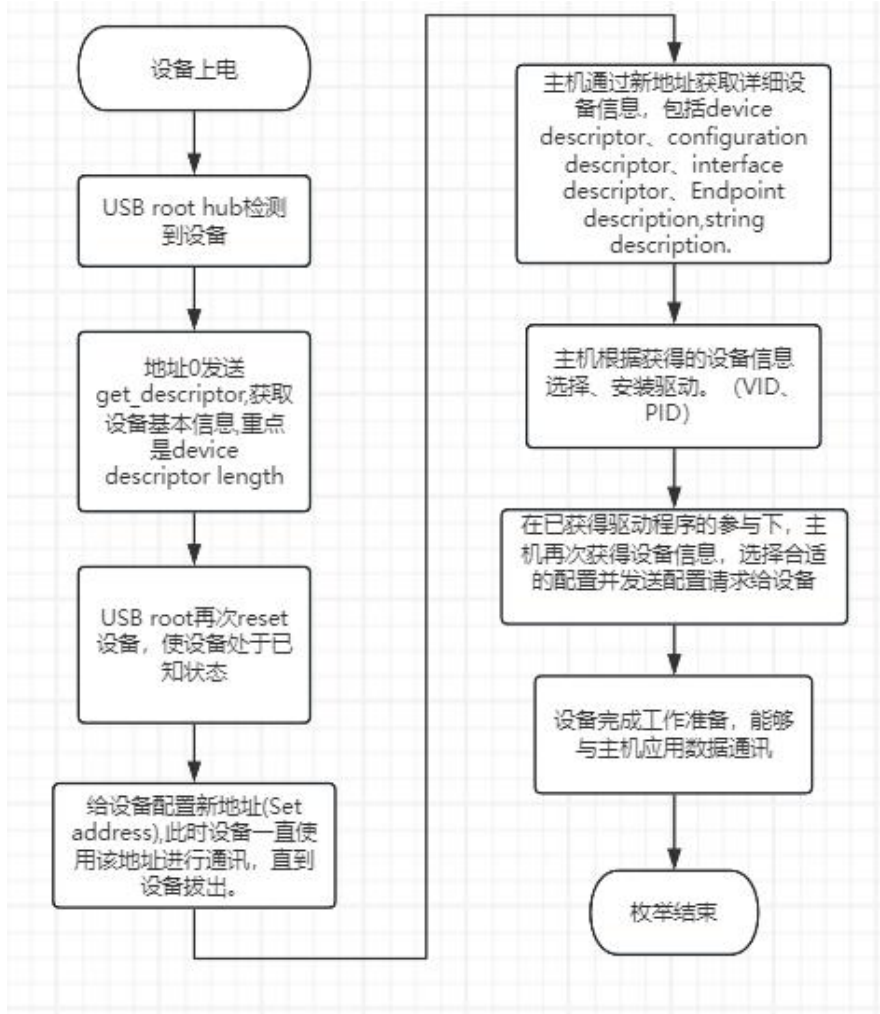
USB gadget 驱动架构

USB gadget 代码路径:

public5 > t-alps-release-r0.mp1-V5 > kernel-4.19 > drivers > usb > gadget > function			
名称	修改日期	类型	大小
f_printer.c	2024/12/23 11:28	sourceinsight.c_file	
f_hid.c	2024/12/19 17:15	sourceinsight.c_file	
f_printer-org.c	2024/11/27 18:49	sourceinsight.c_file	
rndis.c	2024/11/21 22:32	sourceinsight.c_file	
f_accessory.c	2024/11/13 15:06	sourceinsight.c_file	
f_acm.c	2024/11/13 15:06	sourceinsight.c_file	
f_audio_source.c	2024/11/13 15:06	sourceinsight.c_file	
f_ecm.c	2024/11/13 15:06	sourceinsight.c_file	
f_eem.c	2024/11/13 15:06	sourceinsight.c_file	
f_fs.c	2024/11/13 15:06	sourceinsight.c_file	
f_loopback.c	2024/11/13 15:06	sourceinsight.c_file	
f_mass_storage.c	2024/11/13 15:06	sourceinsight.c_file	
f_mass_storage.h	2024/11/13 15:06	H 文件	
f_midi.c	2024/11/13 15:06	sourceinsight.c_file	
f_mtp.c	2024/11/13 15:06	sourceinsight.c_file	
f_mtp.h	2024/11/13 15:06	H 文件	
f_ncm.c	2024/11/13 15:06	sourceinsight.c_file	
f_obex.c	2024/11/13 15:06	sourceinsight.c_file	
f_phonet.c	2024/11/13 15:06	sourceinsight.c_file	
f_ptp.c	2024/11/13 15:06	sourceinsight.c_file	
f_rndis.c	2024/11/13 15:06	sourceinsight.c_file	
f_serial.c	2024/11/13 15:06	sourceinsight.c_file	
f_sourceink.c	2024/11/13 15:06	sourceinsight.c_file	
f_subset.c	2024/11/13 15:06	sourceinsight.c_file	
f_tcm.c	2024/11/13 15:06	sourceinsight.c_file	
f_uac1.c	2024/11/13 15:06	sourceinsight.c_file	
f_uac1_legacy.c	2024/11/13 15:06	sourceinsight.c_file	
f_uac2.c	2024/11/13 15:06	sourceinsight.c_file	
f_uvc.c	2024/11/13 15:06	sourceinsight.c_file	

N4 设备(android):

USB 控制器作为 Device 时，gadget 驱动框架分为 5 层。最上层的是 Gadget Function 驱动，代表了具体设备的驱动，对应我们项目，就是 USB printer、HID 通讯类设备驱动。接下来是 Gadget Funcation API 层，该层是一个抽象层，向上和向下提供统一的 API。Composite 层是一个可选的中间层，可通过一种配置或多种配置支持多种功能的复合设备。目前 N4 使用基于 Composite 和 configs 来实现 USB gadget printer+HID 功能。UDC 驱动直接访问硬件，控制 USB 设备与 USB 主机之间的通信。USB 设备控制器通过 USB 线缆连接 USB 主机控制器（上位机），负责 USB 数据的发送和接收。

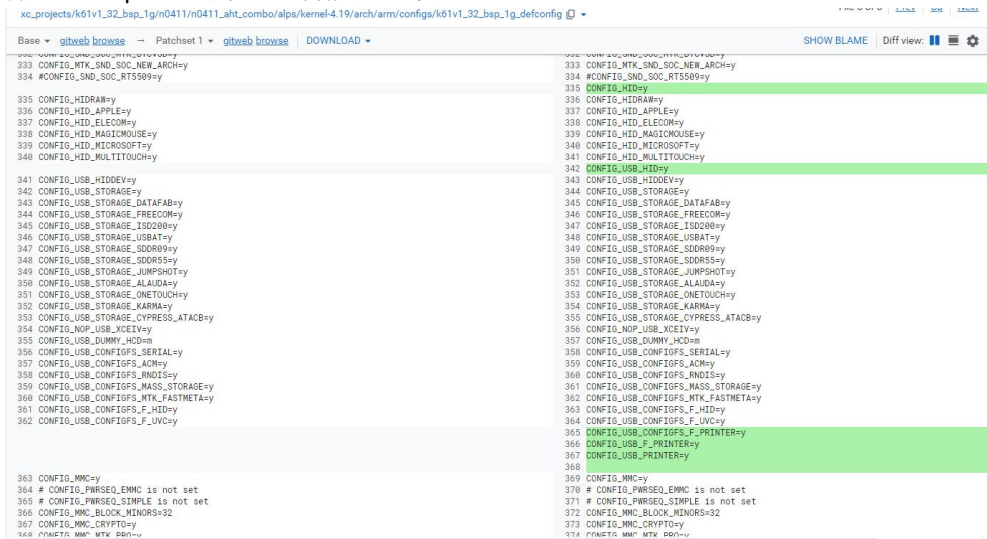
USB 设备枚举流程:**上位机（windows）：**

USB 控制器作为主控，枚举到 N4 为 USB 打印机，USB Host 通过 usb core 接口下发打印数据到 USB 设备控制器，打通了 host 和 device 间数据通路，Linux 内核系统本身既包含 USB 设备主机侧驱动，又包含 USB 设备从机侧驱动（即 USB Gadget 驱动），所以形成一个完整的 USB 驱动框架闭环。

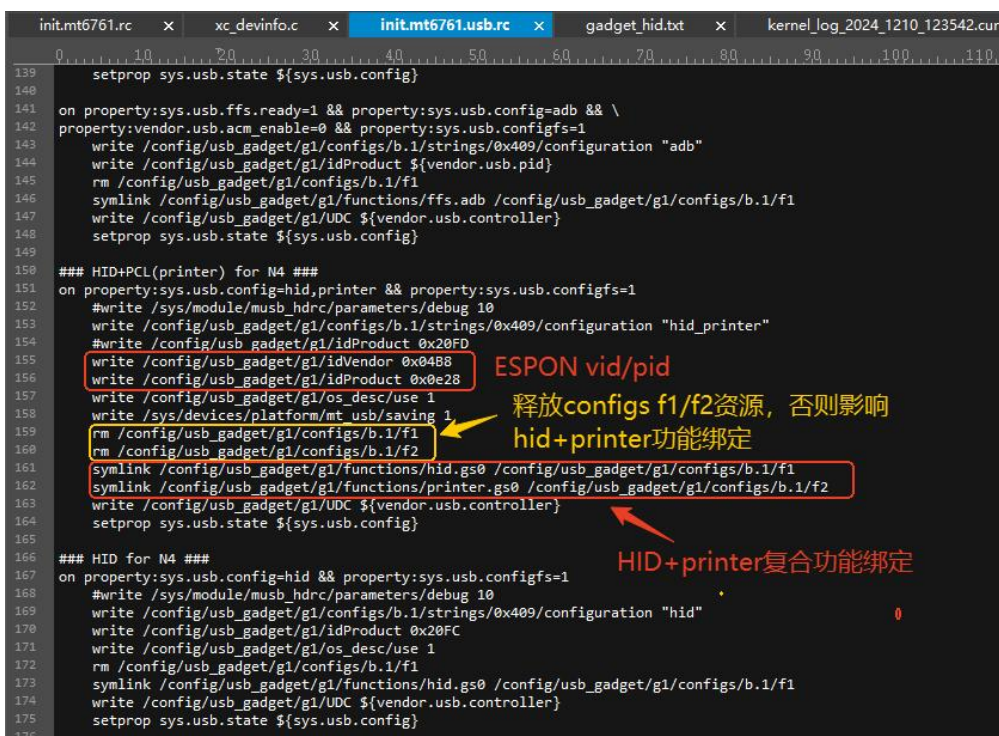
4 驱动开发

4.1 HID+printer 复合功能配置

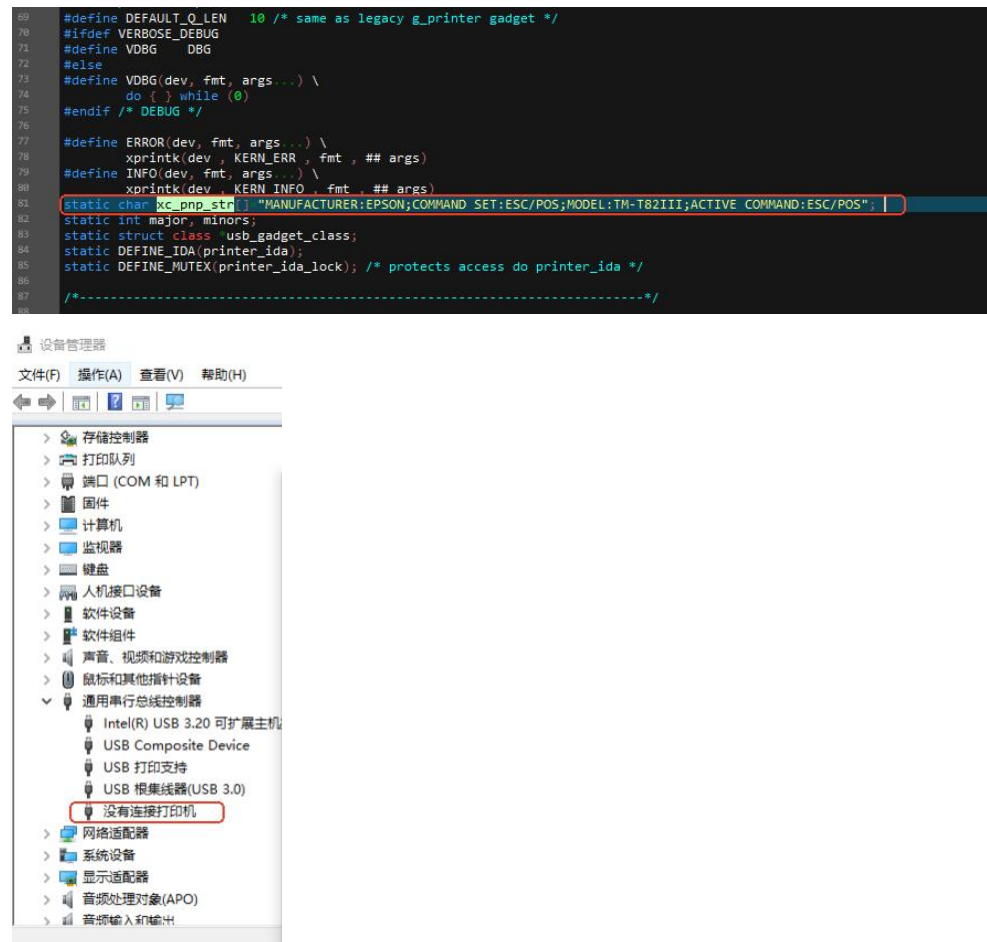
a、开启 HID、printer 的 USB 功能配置项



b、在 init.mt6761.usb.rc 增加 USB HID+printer 复合功能配置，打印设备 VID/PID 有要求，上位机 APP 检测打印设备会通过 VID/PID 来过滤打印设备，现在 N4 配置是 EPSON 的打印机 VID/PID，否则上位机 APP 检测不到打印设备。



c、根据 USB 设备的 pnp_string（Plug and Play 字符串）规范，pnp_string 字符串通常包含设备的厂商、命令集和型号等信息。这些信息可以帮助操作系统识别设备，并为其提供适当的驱动支持。需要在 g_printer.c 驱动中正确配置，否则设备枚举后无法关联打印驱动，PC 检测为“未连接打印机”。



d、定制 HID 设备为键盘设备，需要在 g_hid.c 中配置 HID report descriptor 为键盘类报告符格式，否则 PC 枚举到 HID 设备是带感叹号设备，无法识别标准键盘。

```

+/* hid descriptor for a keyboard */
+static struct hid_func_descriptor my_hid_data = {
+    .subclass      = 0, /* No subclass */
+    .protocol       = 1, /* Keyboard */
+    .report_length  = 8,
+    .report_desc_length = 63,
+    .report_desc    = {
+        0x05, 0x01, /* USAGE_PAGE (Generic Desktop) */
+        0x09, 0x06, /* USAGE (Keyboard) */
+        0xa1, 0x01, /* COLLECTION (Application) */
+        0x05, 0x07, /* USAGE_PAGE (Keyboard) */
+        0x19, 0xe0, /* USAGE_MINIMUM (Keyboard LeftControl) */
+        0x29, 0xe7, /* USAGE_MAXIMUM (Keyboard Right GUI) */
+        0x15, 0x00, /* LOGICAL_MINIMUM (0) */
+        0x25, 0x01, /* LOGICAL_MAXIMUM (1) */
+        0x75, 0x01, /* REPORT_SIZE (1) */
+        0x95, 0x08, /* REPORT_COUNT (8) */
+        0x81, 0x02, /* INPUT (Data,Var,Abs) */
+        0x95, 0x01, /* REPORT_COUNT (1) */
+        0x75, 0x08, /* REPORT_SIZE (8) */
+        0x81, 0x03, /* INPUT (Cnst,Var,Abs) */
+        0x95, 0x05, /* REPORT_COUNT (5) */
+        0x75, 0x01, /* REPORT_SIZE (1) */
+        0x05, 0x08, /* USAGE_PAGE (LEDs) */
+        0x19, 0x01, /* USAGE_MINIMUM (Num Lock) */
+        0x29, 0x05, /* USAGE_MAXIMUM (Kana) */
+        0x91, 0x02, /* OUTPUT (Data,Var,Abs) */
+        0x95, 0x01, /* REPORT_COUNT (1) */
+        0x75, 0x03, /* REPORT_SIZE (3) */
+        0x91, 0x03, /* OUTPUT (Cnst,Var,Abs) */
+        0x95, 0x06, /* REPORT_COUNT (6) */
+        0x75, 0x08, /* REPORT_SIZE (8) */
+        0x15, 0x00, /* LOGICAL_MINIMUM (0) */
+        0x25, 0x65, /* LOGICAL_MAXIMUM (101) */
+        0x05, 0x07, /* USAGE_PAGE (Keyboard) */
+        0x19, 0x00, /* USAGE_MINIMUM (Reserved) */
+        0x29, 0x65, /* USAGE_MAXIMUM (Keyboard Application) */
+        0x81, 0x00, /* INPUT (Data,Ary,Abs) */
+        0xc0 /* END_COLLECTION */
+    }
+};

```

```

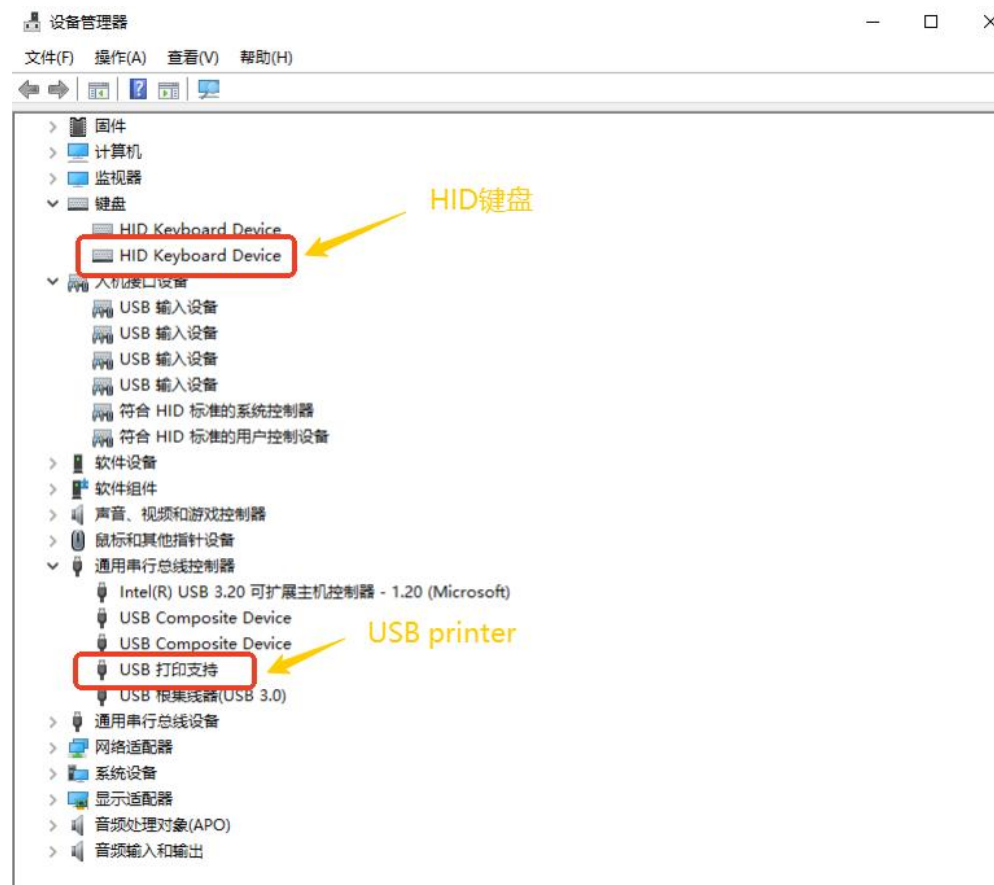
@@ -1096,6 +1139,12 @@ static struct usb_function *hidg_alloc(struct usb_function_instance *fi)
{
    mutex_lock(&opts->lock);
    ++opts->refcnt;
    /*Add keyboard HID function*/
    opts->subclass = my_hid_data.subclass;
    opts->protocol = my_hid_data.protocol;
    opts->report_length = my_hid_data.report_length;
    opts->report_desc_length = my_hid_data.report_desc_length;
    opts->report_desc = my_hid_data.report_desc;

    hidg->minor = opts->minor;
    hidg->bInterfaceSubClass = opts->subclass;
}

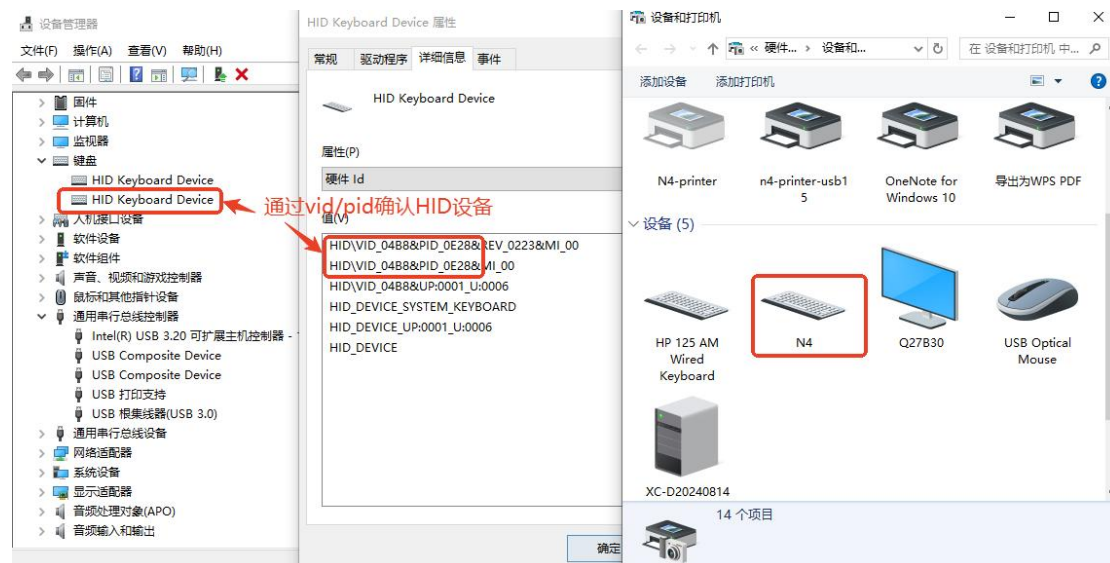
```

4.2 驱动调试

完成 N4 侧 gadget 驱动配置, PC 端识别到 N4 设备为 USB 打印设备和 HID 键盘, 如下图。



通过查看 `init.mt6761.usb.rc` 中定义复合设备的 VID/PID 来确认 N4 模拟是哪个 HID 键盘。



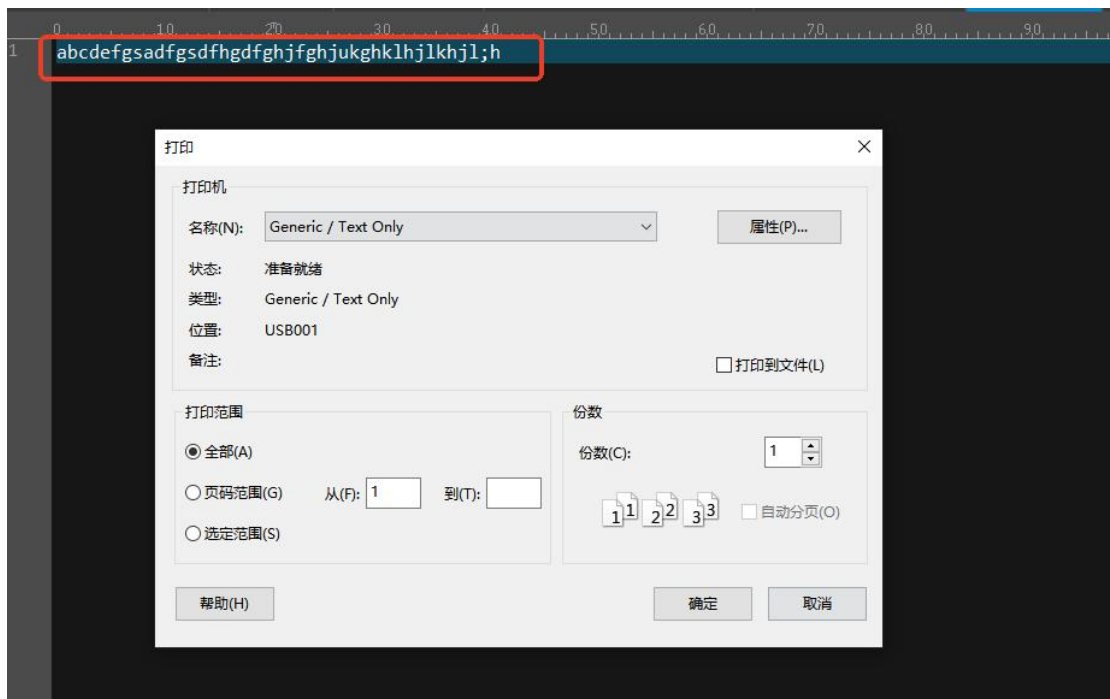
根据打印设备占用 USB 端口号,安装打印机驱动。



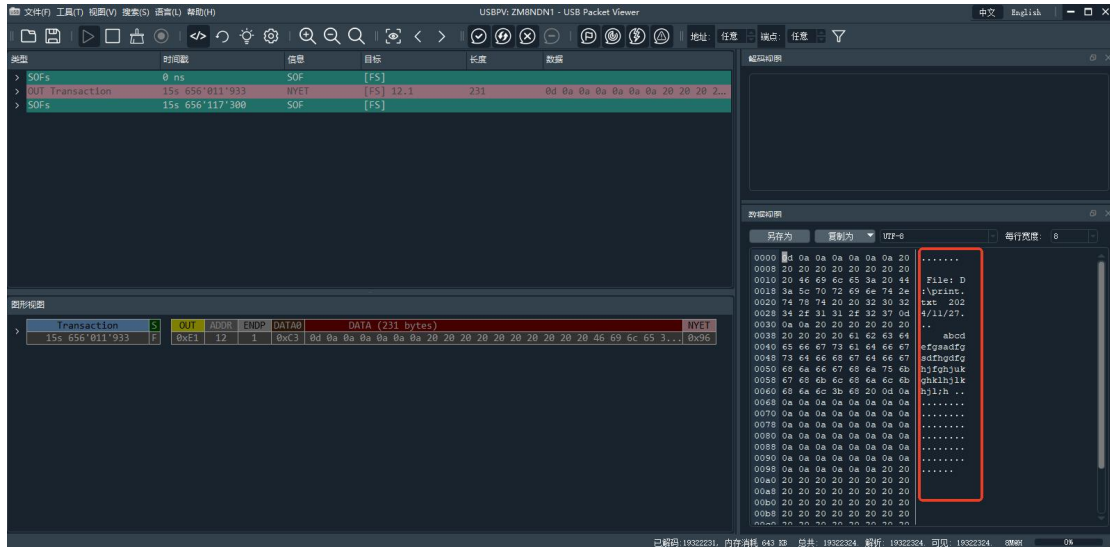
卡点问题:

测试 PC 打印, N4 侧收不到打印数据, 通过 USB 协议分析仪, 抓取打印过程中 USB 总线上数据, 发现 PC 有发出打印数据, N4 侧 gadget g_printer 驱动收不到 OUT 端点数据。

a、PC 下发的打印信息

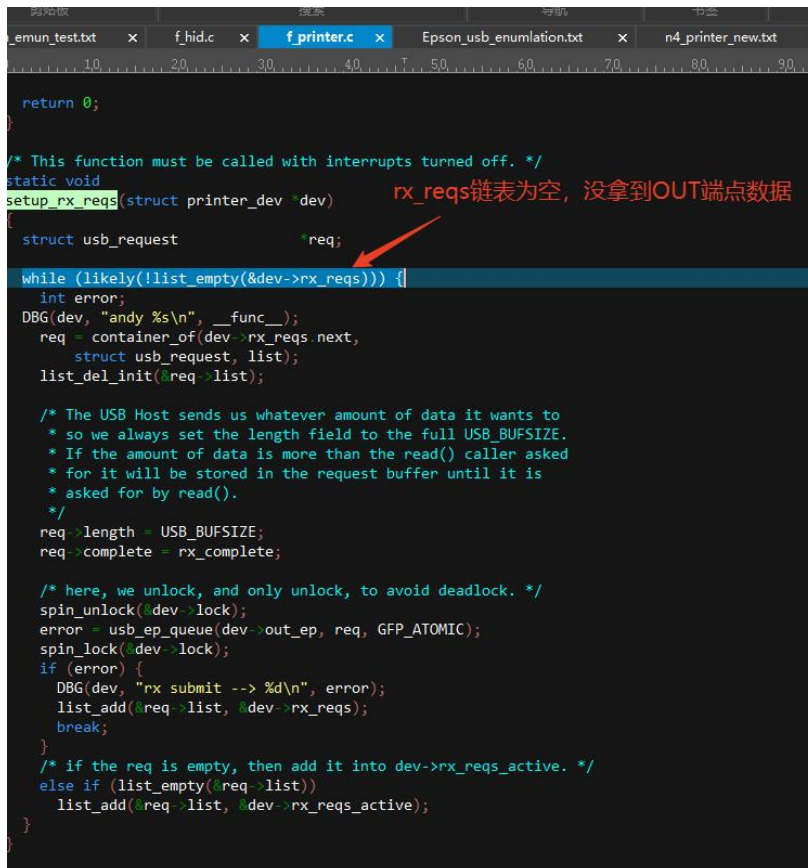


b、抓到 USB OUT 端点上打印数据

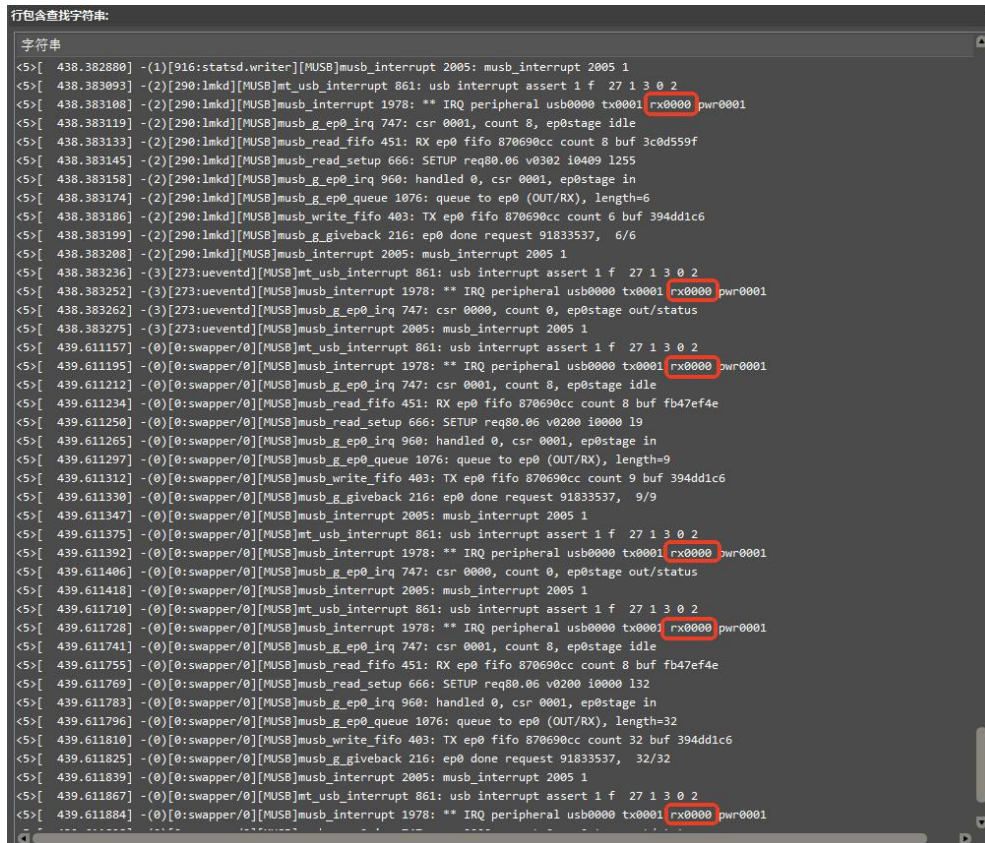


问题分析:

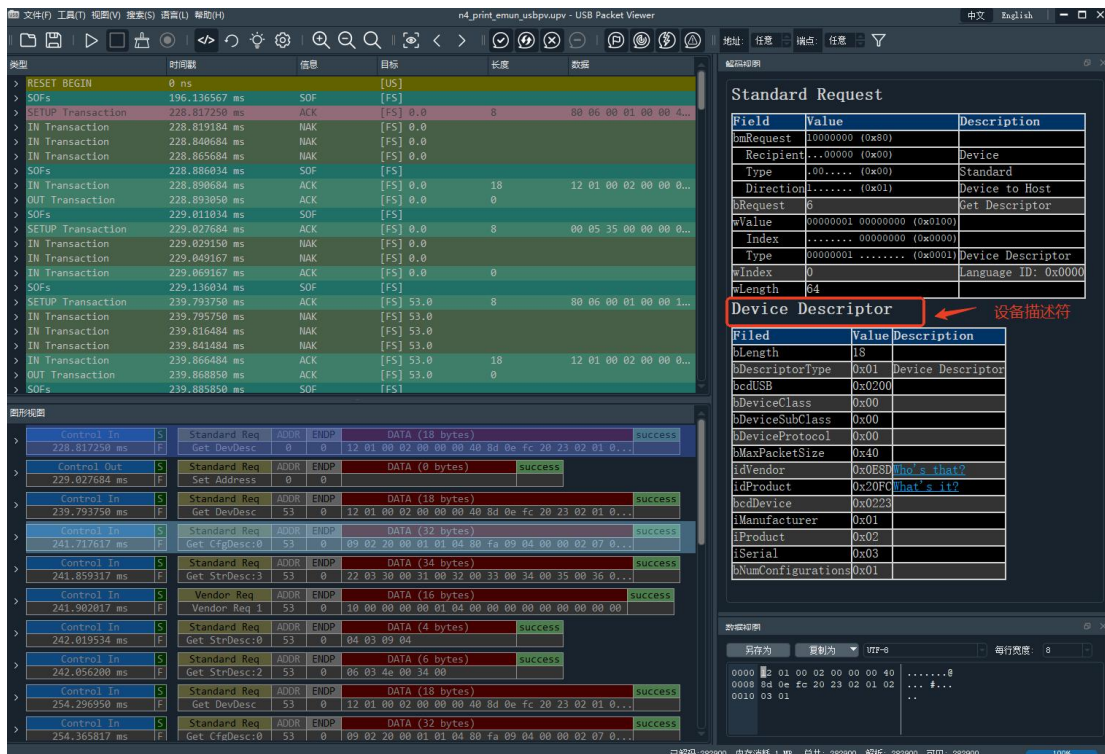
跟踪 g_printer 驱动流程，编写测试 DEMO(见 printer_test.c)，通过 poll 接口，轮询读打印通道数据，发现调用 setup_rx_reqs() 获取 USB OUT 端点数据时，判断条件没满足，流程没走进（见下面标红部分），读不到 OUT EP 数据，由于 USB endpoint 端点收数据是平台 udc 驱动实现的，进一步分析平台 musb core 驱动层数据交互情况，发现 MUSB 驱动层 usb_rx 中断没触发（见附上 log）。



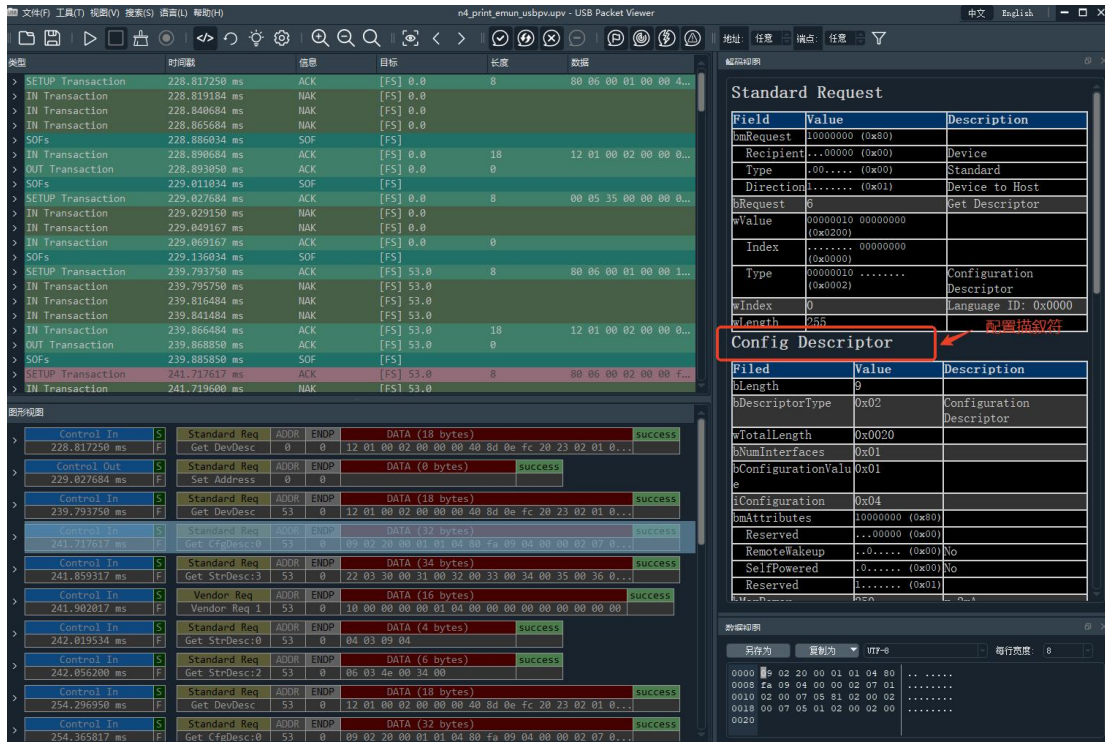
没监测到 USB RX 中断:



同步分析 USB 逻辑分析仪抓取枚举过程，数据正常。
设备描述符：



配置描述符：

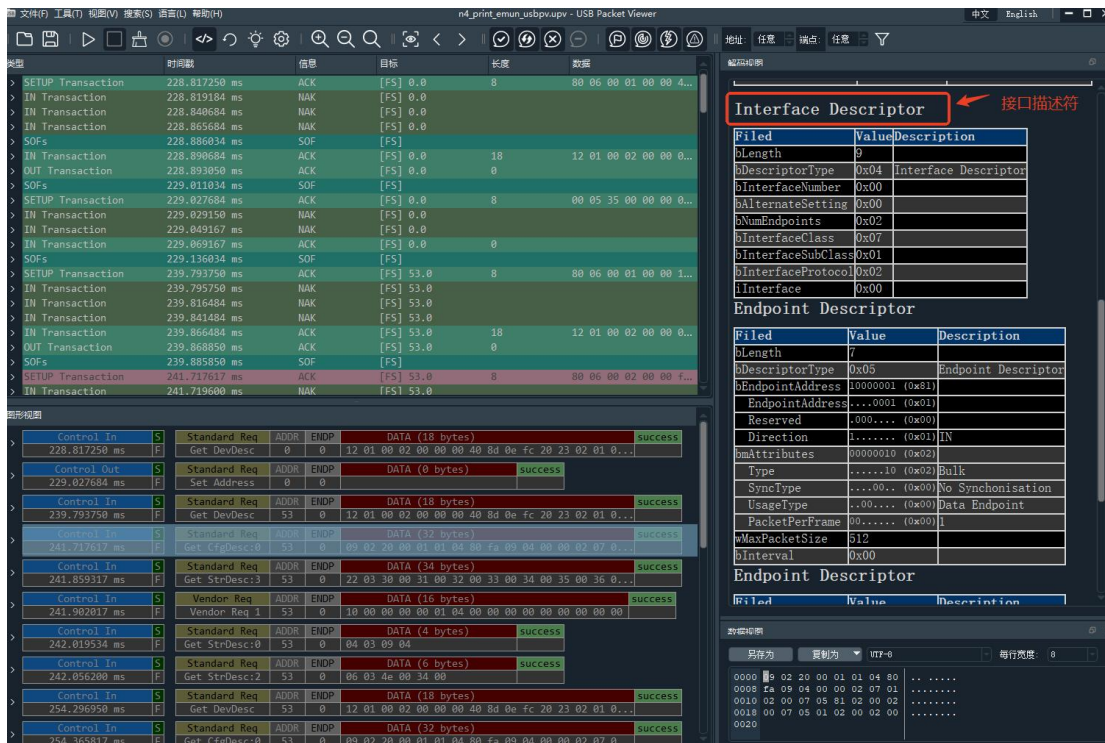


The screenshot shows the USB Packet Viewer interface. The main window displays a list of transactions. The right pane shows the details of a selected packet, which is a Standard Request (bRequest = 0x00, wValue = 0x000000, wIndex = 0). The packet is identified as a Configuration Descriptor (bDescriptorType = 0x02). The details pane shows the following fields:

Field	Value	Description
bRequest	0x000000 (0x00)	
Recipient	...000000 (0x00)	Device
Type	...00.... (0x00)	Standard
Direction	1..... (0x01)	Device to Host
bRequest	0	Get Descriptor
wValue	00000010 00000000 (0x0200)	
Index	 00000000 (0x0000)	
Type	00000010 (0x0002)	Configuration Descriptor
wIndex	0	Language ID: 0x0000
Length	255	

The packet data is shown in hexadecimal and ASCII format at the bottom of the details pane.

接口描述符:

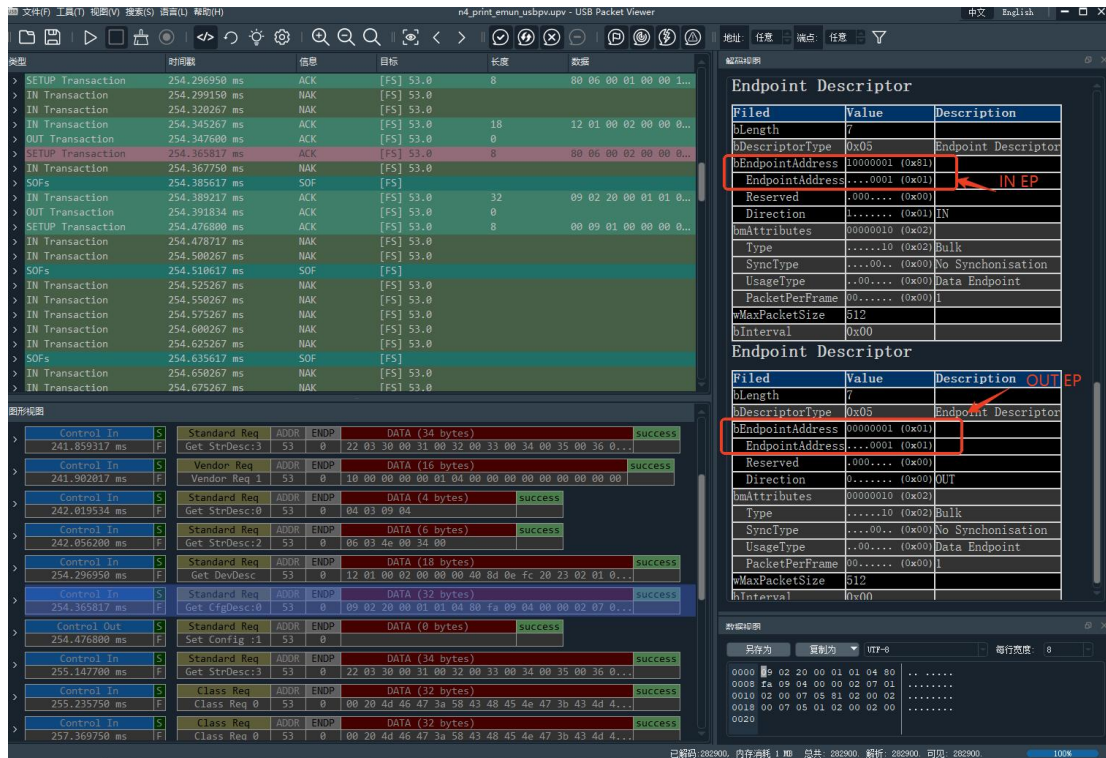


The screenshot shows the USB Packet Viewer interface. The main window displays a list of transactions. The right pane shows the details of a selected packet, which is an Interface Descriptor (bRequest = 0x00, wValue = 0x000000, wIndex = 0). The packet is identified as an Interface Descriptor (bDescriptorType = 0x04). The details pane shows the following fields:

Field	Value	Description
bLength	9	
bDescriptorType	0x04	Interface Descriptor
bInterfaceNumber	0x00	
bAlternateSetting	0x00	
bNumEndpoints	0x02	
bInterfaceClass	0x07	
bInterfaceSubClass	0x01	
bInterfaceProtocol	0x02	
iInterface	0x00	

The packet data is shown in hexadecimal and ASCII format at the bottom of the details pane.

端点描述符:



原因定位:

进一步跟踪平台 USB ep 端点数据控制逻辑（见《usb_printer_data_flow.pdf》），当 USB 做 device 设备，虚拟 gadget 功能设备和 USB UDC 是通过 usb request 方式进行数据通讯，printer 设备在 gadget bind 后，需要申请 usb_request 链表资源关联到平台 MUSB EP 的 usb req 链表上，否则 usb 端点中断不会触发。通过查看 USB printer 协议得知，需要配置 opts->q_len 来申请 usb request 链表资源，而 gadget f_printer 驱动默认该值为 0，导致没有申请平台 USB EP 空间，没法触发 USB EP 中断接受数据。

```

1363 static struct usb_function_instance *gprinter_alloc_inst(void)
1364 {
1365     struct f_printer_opts *opts;
1366     struct usb_function_instance *ret;
1367     int status = 0;
1368
1369     opts = kzalloc(sizeof(*opts), GFP_KERNEL);
1370     if (!opts)
1371         return ERR_PTR(-ENOMEM);
1372
1373     mutex_init(&opts->lock);
1374     opts->func_inst->free_func_inst = gprinter_free_inst;
1375     ret = &opts->func_inst;
1376
1377     /* Make sure q_len is initialized, otherwise the bound device can't support read/write! */
1378     opts->q_len = DEFAULT_Q_LEN;
1379
1380     mutex_lock(&printer_ida_lock);
1381
1382     if (ida_is_empty(&printer_ida)) {
1383         status = gprinter_setup(PRINTER_MINORS);
1384         if (status) {
1385             ret = ERR_PTR(status);
1386             kfree(opts);
1387             goto unlock;
1388         }
1389     }
1390
1391     opts->minor = gprinter_get_minor();
1392     if (opts->minor < 0) {
1393         ret = ERR_PTR(opts->minor);
1394         kfree(opts);
1395         if (ida_is_empty(&printer_ida))
1396             gprinter_cleanup();
1397         goto unlock;
1398     }
1399     config_group_init_type_name(&opts->func_inst->group, "",
1400                                &printer_func_type);
1401
1402     unlock:
1403     mutex_unlock(&printer_ida_lock);
1404     return ret;
1405 }
1406

```



```

1093 dev_err(&cdev, "can't autoconfigure on %s\n",
1094         cdev.gadget.name);
1095 return ENODEV;
1096 }
1097
1098 out_ep = usb_ep_autoconfig(&cdev->gadget, &fs_ep_out_desc);
1099 if (!out_ep)
1100     goto autoconf_fail;
1101
1102 /* assumes that all endpoints are dual-speed */
1103 hs_ep_in_desc.bEndpointAddress = fs_ep_in_desc.bEndpointAddress;
1104 hs_ep_out_desc.bEndpointAddress = fs_ep_out_desc.bEndpointAddress;
1105 ss_ep_in_desc.bEndpointAddress = fs_ep_in_desc.bEndpointAddress;
1106 ss_ep_out_desc.bEndpointAddress = fs_ep_out_desc.bEndpointAddress;
1107
1108 ret = usb_assign_descriptors(f, fs_printer_function,
1109                             hs_printer_function, ss_printer_function, NULL);
1110 if (ret)
1111     return ret;
1112
1113 dev->in_ep = in_ep;
1114 dev->out_ep = out_ep;
1115
1116 ret = -ENOMEM;
1117 for (i = 0; i < dev->qlen; i++) {
1118     req = printer_req_alloc(dev->in_ep, USB_BUFSIZE, GFP_KERNEL);
1119     if (!req)
1120         goto fail_tx_reqs;
1121     list_add(&req->list, &dev->tx_reqs);
1122 }
1123
1124 for (i = 0; i < dev->qlen; i++) {
1125     req = printer_req_alloc(dev->out_ep, USB_BUFSIZE, GFP_KERNEL);
1126     if (!req)
1127         goto fail_rx_reqs;
1128     list_add(&req->list, &dev->rx_reqs);
1129 }
1130
1131 /* Setup the sysfs files for the printer gadget. */
1132 devt = MKDEV(major, dev->minor);
1133 pdev = device_create(&usb_gadget_class, NULL, devt,
1134                     NULL, "g_printer%d", dev->minor);
1135 if (IS_ERR(pdev)) {
1136     ERROR(dev, "Failed to create device: g_printer\n");
1137     ret = PTR_ERR(pdev);
1138     goto fail_rx_reqs;
1139 }
1140

```

测试验证:

前提: 开启 WIFI ADB 功能, N4 设备

1) 验证 printer 功能

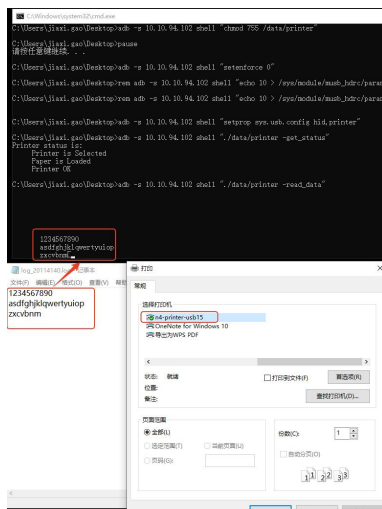
- 编译 printer_test.c 生成打印测试 DEMO (printer) .
- 配置 printer 工具路径, 执行 switch_hid_printer.bat 测试脚本, 显示下图等 PC 打印数据。

```

C:\Windows\system32\cmd.exe
C:\Users\jiaxi.gao\Desktop>adb -s 10.10.94.102 root
adb is already running as root
C:\Users\jiaxi.gao\Desktop>adb -s 10.10.94.102 remount
remount succeeded
C:\Users\jiaxi.gao\Desktop>adb -s 10.10.94.102 push D:\work\project\printer\printer_test\printer /data/printer
D:\work\project\printer\printer_test\printer: 1 file pushed. 0 skipped. 19.3 KB/s (1108 bytes in 0.001s)
C:\Users\jiaxi.gao\Desktop>adb -s 10.10.94.102 shell "chmod 755 /data/printer"
C:\Users\jiaxi.gao\Desktop>pause
请按任意键继续. . .
C:\Users\jiaxi.gao\Desktop>adb -s 10.10.94.102 shell "setenforce 0"
C:\Users\jiaxi.gao\Desktop>rem adb -s 10.10.94.102 shell "echo 10 > /sys/module/musb_hcd/parameters/debug"
C:\Users\jiaxi.gao\Desktop>rem adb -s 10.10.94.102 shell "echo 10 > /sys/module/musb_hcd/parameters/mtk_gmu_dbg_level"
C:\Users\jiaxi.gao\Desktop>adb -s 10.10.94.102 shell "setprop sys.usb.config hid,printer"
C:\Users\jiaxi.gao\Desktop>adb -s 10.10.94.102 shell "/data/printer -get_status"
Printer status is:
Printer is Selected
Paper is Loaded
Printer OK
C:\Users\jiaxi.gao\Desktop>adb -s 10.10.94.102 shell "/data/printer -read_data"

```

- PC 触发文本打印, N4 设备接收到打印数据



2) 验证 HID 键盘设备

- 编译 hid_test.c 生成打印测试 DEMO (hid)
- N4 设备通过 USB OTG 连接上位机 (android 系统)，上位机任意开启可输入信息应用界面。
- 配置 hid 工具路径，执行 hid-keyboard.bat 测试脚本，输入按键（如：a b c d e --left-shift），模拟键盘 A B C D E 行为。

```

C:\Windows\system32\cmd.exe - adb shell
127|N4:/ # clear
N4:/ # ./data/hid /dev/hidg0 keyboard
keyboard options:
    --hold
    --left-ctrl
    --right-ctrl
    --left-shift
    --right-shift
    --left-alt
    --right-alt
    --left-meta
    --right-meta

keyboard values:
[a-z] or
--return          --esc
--bckspc          --tab
--spacebar        --caps-lock
--f1              --f2
--f3              --f4
--f5              --f6
--f7              --f8
--f9              --f10
--f11             --f12
--insert          --home
--pageup          --del
--end             --pagedown
--right           --left
--down            --kp-enter
--up              --num-lock
  
```

```

C:\Windows\system32\cmd.exe
C:\Users\jiayi.gao\Desktop>set ip_addr=10.10.94.102
C:\Users\jiayi.gao\Desktop>set printer_path=D:\work\project\printer\printer_test
C:\Users\jiayi.gao\Desktop>set hid_dev=/dev/hidg0
C:\Users\jiayi.gao\Desktop>adb disconnect 10.10.94.102
disconnected 10.10.94.102
C:\Users\jiayi.gao\Desktop>rem adb -s 0123456789ABCDEF tcpip 5555
C:\Users\jiayi.gao\Desktop>adb connect 10.10.94.102
connected to 10.10.94.102:5555
C:\Users\jiayi.gao\Desktop>adb -s 10.10.94.102 root
adb is already running as root
C:\Users\jiayi.gao\Desktop>adb -s 10.10.94.102 remount
remount succeeded
C:\Users\jiayi.gao\Desktop>adb -s 10.10.94.102 push D:\work\project\printer\printer_test\hid /data/hid
D:\work\project\printer\printer_test\hid: 1 file pushed, 0 skipped. 4.8 MB/s (12108 bytes in 0.002s)
C:\Users\jiayi.gao\Desktop>adb -s 10.10.94.102 shell "chmod 755 /data/hid"
C:\Users\jiayi.gao\Desktop>adb -s 10.10.94.102 shell "./data/hid /dev/hidg0 keyboard"
a b c d e --left-shift
  
```

模拟输入 A B C D E 按键

- 上位机 APP 的输入界面，会正常响应对应按键动作。

